

IHK SQL Handbuch 2025 – AP2 FIAE

Druckfreundlich · Erklärung · Pseudocode in SQL · IHK-Stil-Beispiele · Tipps & Fehlerfallen

Inhalt

1. Grundlagen: SELECT, WHERE, ORDER BY
 2. Filter: AND, OR, NOT, BETWEEN, IN, LIKE, NULL
 3. Joins: INNER JOIN, LEFT JOIN, Mehrfach-Joins
 4. Aggregatfunktionen
 5. GROUP BY & HAVING
 6. Einfache Subselects
 7. Praxisbeispiele (IHK-Stil)
 8. Fehlerfallen & Best Practices
 9. Kurzreferenz
-

1) Grundlagen: SELECT, WHERE, ORDER BY

SELECT ... FROM ...

```
SELECT spalte1, spalte2  
FROM tabelle;
```

Beispiel: `SELECT name, preis FROM artikel;`

WHERE (Zeilen filtern)

```
SELECT name, preis  
FROM artikel  
WHERE preis > 100;
```

ORDER BY (Sortieren)

```
SELECT name, preis  
FROM artikel  
ORDER BY preis DESC, name ASC;
```

Spalten-Alias & Berechnungen

```
SELECT name, preis, preis * 1.19 AS preis_mit_mwst  
FROM artikel;
```

Zeilen begrenzen

```
-- Standard SQL / viele Systeme  
SELECT *  
FROM artikel  
ORDER BY preis DESC  
FETCH FIRST 10 ROWS ONLY;
```

```
-- MySQL / MariaDB / SQLite
SELECT * FROM artikel ORDER BY preis DESC LIMIT 10;
```

2) Filter: AND, OR, NOT, BETWEEN, IN, LIKE, NULL

Vergleichsoperatoren

```
=, <>, >, <, >=, <=
```

```
SELECT name FROM kunden WHERE plz = '20095';
```

AND / OR / NOT

```
SELECT name
FROM artikel
WHERE (preis > 50 AND preis <= 200)
      OR (kategorie = 'Buch' AND NOT gelöscht);
```

BETWEEN

```
SELECT name FROM kunden
WHERE plz BETWEEN '20000' AND '29999';
```

IN

```
SELECT name FROM artikel
WHERE kategorie IN ('Buch', 'Software', 'Zubehör');
```

LIKE (Platzhalter)

```
SELECT name FROM kunden
WHERE name LIKE 'M%'; -- beginnt mit M
-- '_' steht für ein einzelnes Zeichen
```

NULL prüfen

```
SELECT name FROM kunden
WHERE telefon IS NULL;    -- kein Telefon eingetragen
```

Merke: Mit = NULL findet man keine NULLs. Immer IS NULL / IS NOT NULL verwenden.

3) Joins: INNER JOIN, LEFT JOIN, Mehrfach-Joins

INNER JOIN (Standard)

```
SELECT k.name, b.bestellnr, b.datum
FROM kunden k
JOIN bestellung b ON b.kundennr = k.kundennr;
```

Mehrere Joins

```
SELECT k.name, b.bestellnr, a.name AS artikel, p.menge
FROM bestellung b
JOIN kunden k ON k.kundennr = b.kundennr
JOIN position p ON p.bestellnr = b.bestellnr
JOIN artikel a ON a.artnr = p.artnr;
```

LEFT JOIN (zeige auch Datensätze ohne Treffer rechts)

```
SELECT k.name, b.bestellnr
FROM kunden k
LEFT JOIN bestellung b ON b.kundennr = k.kundennr;
```

Typischer IHK-Fall: „Zeige alle Kunden, auch wenn sie keine Bestellung haben.“ → **LEFT JOIN**.

JOIN mit zusätzlichen Bedingungen

```
SELECT k.name, b.bestellnr
FROM kunden k
JOIN bestellung b ON b.kundennr = k.kundennr AND b.status = 'offen';
```

4) Aggregatfunktionen

Zählen, Summieren, Durchschnitt

```
SELECT COUNT(*)      AS anzahl_bestellungen FROM bestellung;  
SELECT SUM(preis)     AS umsatz              FROM artikel;  
SELECT AVG(preis)     AS schnitt             FROM artikel;  
SELECT MIN(preis)     AS guenstigster        FROM artikel;  
SELECT MAX(preis)     AS teuerster          FROM artikel;
```

Mit Bedingung

```
SELECT COUNT(*) AS anzahl  
FROM bestellung  
WHERE datum >= '2025-01-01';
```

Merke: Aggregatfunktionen ignorieren NULL (außer COUNT(*) zählt Zeilen).

5) GROUP BY & HAVING

Gruppieren

```
SELECT k.kundennr, COUNT(*) AS anzahl  
FROM bestellung b  
JOIN kunden k ON k.kundennr = b.kundennr  
GROUP BY k.kundennr;
```

Mehrere Spalten gruppieren

```
SELECT k.kundennr, k.plz, COUNT(*) AS anzahl  
FROM bestellung b  
JOIN kunden k ON k.kundennr = b.kundennr  
GROUP BY k.kundennr, k.plz;
```

HAVING (Bedingung auf Gruppen)

```
SELECT k.kundennr, COUNT(*) AS anzahl
FROM bestellung b
JOIN kunden k ON k.kundennr = b.kundennr
GROUP BY k.kundennr
HAVING COUNT(*) > 5;
```

Fehler: HAVING ist für Bedingungen nach dem Gruppieren. Zeilen-Bedingungen gehören in WHERE.

6) Einfache Subselects (falls gefordert)

IN-Subselect

```
SELECT name
FROM kunden
WHERE kundennr IN (
    SELECT kundennr FROM bestellung WHERE datum >= '2025-01-01'
);
```

Vergleich mit Aggregat

```
SELECT name, preis
FROM artikel
WHERE preis > (
    SELECT AVG(preis) FROM artikel
);
```

Merke: Subselects sparsam einsetzen – die IHK nutzt meist einfache Varianten.

7) Praxisbeispiele (IHK-Stil)

a) Kunden aus Hamburg mit > 5 Bestellungen

```
SELECT k.kundennr, k.name, COUNT(*) AS anzahl
FROM kunden k
JOIN bestellung b ON b.kundennr = k.kundennr
WHERE k.ort = 'Hamburg'
GROUP BY k.kundennr, k.name
HAVING COUNT(*) > 5
ORDER BY anzahl DESC;
```

b) Artikel mit Durchschnittspreis pro Kategorie

```
SELECT kategorie, AVG(preis) AS avg_preis
FROM artikel
GROUP BY kategorie
ORDER BY avg_preis DESC;
```

c) Kunden ohne Bestellung

```
SELECT k.kundennr, k.name
FROM kunden k
LEFT JOIN bestellung b ON b.kundennr = k.kundennr
WHERE b.bestellnr IS NULL;
```

d) Meistverkaufte Artikel (Top-3)

```
SELECT a.artnr, a.name, SUM(p.menge) AS gesamt
FROM position p
JOIN artikel a ON a.artnr = p.artnr
```

```
GROUP BY a.artnr, a.name  
ORDER BY gesamt DESC  
FETCH FIRST 3 ROWS ONLY;    -- oder: LIMIT 3
```

8) Fehlerfallen & Best Practices

- **NULL:** Mit = NULL findet man nichts → IS NULL nutzen.
- **GROUP BY:** Alle nicht-aggregierten Spalten in GROUP BY aufnehmen.
- **WHERE vs. HAVING:** Zeilenbedingung in WHERE, Gruppenbedingung in HAVING.
- **JOINS:** Prüfen, ob *alle* Zeilen der linken Tabelle benötigt werden → ggf. LEFT JOIN.
- **ORDER BY:** Bei Rankings Sortierung explizit angeben (ASC/DESC).
- **Datumsformate:** Einheitlich schreiben (z. B. '2025-10-18').

9) Kurzreferenz

```
SELECT spalten  
FROM tabelle  
JOIN t2 ON ...  
WHERE bedingung  
GROUP BY spalten  
HAVING gruppenbedingung  
ORDER BY spalte ASC|DESC;
```

Druck: Browser → Drucken → Als PDF speichern (weißes Layout).